Seq2Seq and Embedding-based Mini-Translators

Joint Final Project NLP and DL - First Progress Report

Elena Ardura, Pablo Díaz, Victoria García and Jimena Monteagudo

April 12, 2024

Project Focus

In order to create a translation machine able to translate phrases from one language to another, we have decided to implement a Sequence-to-Sequence (Seq2Seq) model using a Recurrent Neural Network (RNN). The Seq2Seq architecture consists of two elements: an encoder and a decoder. The latter takes the input sentence in one language, and the former returns its translation to another language. They both include a Long-Short Term Memory neural network, and their respective roles are now described.

Firstly, the encoder takes the input sentence in the first language as a sequence of word embeddings. This means that each word of the sentence previously goes through an embedding layer so that it is represented as a numerical vector. Subsequently, the sequence of word embeddings is fed to the LSTM so that at each time step t, the word embedding x_t and the previous hidden state h^E_{t-1} are processed jointly to generate a new hidden state, h^E_t. The final hidden state of the encoder, h^E_T, acts as a summarized representation of the input sequence.

On the other hand, the decoder takes in a start-of-sequence token (denoted as `<SOS>`) to indicate the beginning of the output sentence, and the initial hidden state of its LSTM is the final hidden state of the encoder, h^E_T. Then, at each time step t a new hidden state h^D_t is produced taking both the previous output token y_{t-1} and the hidden state from the previous time step, h^D_{t-1}. Each hidden state h^D_t is passed through a fully connected layer (which contains a linear layer followed by a softmax activation function) to produce a probability distribution over the vocabulary of the second language. The output token with the highest probability is selected as the predicted token y_t for the current time step t.

During the testing stage, the predicted token y_t is used as the input for the next time step $t+1$. Nevertheless, during the training period the input for the time step $t+1$ is the correct target for the step t. This strategy is called *teacher forcing*, and allows a faster learning with more accurate values. In both cases, the prediction process is repeated until an end-of-sequence token (denoted as `<EOS>`) is generated or a fixed maximum length is reached.

In addition, it should be noted that the data chosen to train the model consists of parallel corpora sets. These are collections of texts (in two languages in our case) in which each sentence corresponds to its translation to another language. Instead of taking Wikipedia articles' titles we finally decided to use the suggested dataset related to the European Parliament since the grammatical structures are more complex and appropriate.

Model thus far

To develop the Seq2Seq architecture, we have created the encoder and decoder elements. They both consist of a layer that creates the embeddings of the word it receives, and a LSTM layer that processes those embeddings and generates the hidden states. Moreover, the decoder includes a linear layer that returns the outputs.

To start with, we created a script for the obtaining and preprocessing of the data. This script loads the data, creates the class objects for each language (each of them with its respective words, creating the correspoding vocabulary), and prepares the data to be assembled into a Dataloader object. This preparation implies the addition to each sentence of the start and end tokens and the creation of the input and target tensors. Besides, padding is implemented with the *pad_sequence* function from the *torch.nn.utils.rnn* module to ensure that all the sentences in the same batch have the same length (in our case, it is set at 50 words). We made the decision to use this method instead of adding the padding manually to the sentences so that the network does not have to process the padding tokens and the computational cost is reduced (easing faster training).

Once the dataloaders are constructed, it is time to train the model. Initially, as well as training the translation model we were simultaneously training the embeddings of the words in the vocabulary. Notwithstanding, we have finally disregarded this option since it significantly increased the complexity of the model, and it resulted in extremely long training periods. We have chosen the pretrained embeddings included in the *torch.nn* Python library, and this layer is included in the encoder and decoder (as aforementioned).

Lastly, it is worth noting that the selected loss function for the training stage is the Cross Entropy Loss (implemented in Pytorch as *torch.nn.CrossEntropyLoss*), so that the backward propagation can be computed in every step. Conversely, the metric used in the validation and testing phases is the BLEU (Bilingual Evaluation Understudy) score. This metric is widely used in machine translation models because it measures the quality of the generated translations by comparing them with the reference translations. Still, this metric will be detailedly explained in the final paper and in the project presentation given that it is not part of the syllabus of the course. What is more, examples of its use will be provided when the model performs correctly.

Preliminary Results

After carrying out the first training attempts with the implemented model, we concluded that the use of pretrained embeddings was crucial to reduce the cost and complexity of the model. Furthermore, we faced the need to replace the dataset we originally selected so that the information and vocabulary were broader and more complete.

Once we performed these changes, in addition to modifying some of the parameters (especially the batch size), we were finally able to train our first model, which took approximately between 15 and 20 minutes. Nonetheless, when introducing a simple sentence such as *'I am a student'* the obtained output was a sequence of the start token (`<SOS>`) repeated several times. We then changed this sentence and used as input one of the sentences contained in the training data, and yet the result was exactly the same.

As a consequence of these trials, it was confirmed that the probabilities computed both in the encoder and the decoder are incorrect. Therefore, we are currently trying to find the error in the implementation so that the model can be fixed.

Next Steps

Our objective for the following week is, first and foremost, to finish and perfect the model so that the results returned are correct and, if possible, to implement an attention system so that its performance is better. Besides, we intend to control all the problems and exceptions that the implementation may cause (for instance, if the input sentence contains words that are not part of the vocabulary).

Provided we bring about an acceptable model that translates from English to Spanish, we would like to try the translation from English to other languages and, if possible, to implement a transformer in order to compare the obtained results (not only in terms of quality but also computational cost and model complexity). We believe the latter is fairly ambitious but it could be of great help for model analysis and to choose which option is more suitable for machine translation.